

Assignment 4: Machine Learning and Neural Networks

(Upload to Moodle and defend by 08.11.2025)
If the assignment is submitted and defended late:
–20% from each task
(i.e., if everything is completed, the max score = **80 points**)

Task 1. Classification on the Iris dataset (Scikit-learn) 25 баллов

Objective: apply classical classification algorithms and compare them.

Task:

1. Load the Iris dataset from `sklearn.datasets`.
2. Split the data into train/test (80/20).
3. Train the following models:
 - Decision Tree
 - Logistic Regression
 - K-Nearest Neighbors (KNN)
4. For each model, compute the metrics:
 - Accuracy, Precision, Recall, F1-score
 - Plot the ROC curve and compute AUC.
5. Compare the results of the models and make a brief conclusion (which model shows the best balance of metrics).

Steps for completion:

1. **Import and environment fixing**
 - Import `numpy`, `pandas`, `matplotlib`, `sklearn`.
 - Fix `random_state=42` for reproducibility.
2. **Data loading and initial analysis**
 - Load `load_iris()` → obtain X, y, feature/class names.
 - Check array shapes, class balance (`np.bincount(y)`).
3. **Train/test split**
 - Perform `train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)`.
 - Stratification is mandatory (uniform distribution of classes in train/test).
4. **Preprocessing pipelines + models**
 - For Logistic Regression and KNN use feature scaling:
 - `Pipeline([("scaler", StandardScaler()), ("clf", LogisticRegression(...))])`
 - `Pipeline([("scaler", StandardScaler()), ("clf", KNeighborsClassifier(...))])`
 - For Decision Tree, scaling is not required:
 - `Pipeline([("clf", DecisionTreeClassifier(...))])`
5. **(Optional) Light hyperparameter tuning**
 - Mini-GridSearchCV (5-fold):
 - LR: $C \in \{0.1, 1, 10\}$; `penalty="l2"`, `solver="lbfgs"`, `max_iter=1000`
 - KNN: `n_neighbors` $\in \{3, 5, 7, 9\}$

- DT: `max_depth` ∈ {2, 3, 4, None}, `min_samples_split` ∈ {2, 4, 6}
 - Save the best configurations or fix reasonable defaults.
- 6. **Training final models**
 - Train the 3 models on train.
 - Save training time (optional) for future comparison of “speed”.
- 7. **Predictions and test metrics**
 - Obtain `y_pred` and `y_proba/decision_function` (for ROC-AUC).
 - Compute:
 - Accuracy
 - Precision, Recall, F1 (for a multiclass task use `average="macro"` and state this in the report)
 - Classification report (by classes)
 - Confusion matrix (visualize as heatmap)
- 8. **ROC curves and AUC (multiclass case)**
 - Use the One-vs-Rest (OvR) scheme:
 - Binarize y: `label_binarize(y_test, classes=[0,1,2])`
 - Plot ROC for each class (by columns of `predict_proba/decision_function`)
 - Compute macro-AUC (average across classes)
 - On one plot display 3 curves (by classes) for each model or make separate plots per model.
- 9. **Summary results table**
 - Form a table (rows — models, columns — Accuracy, Precision(macro), Recall(macro), F1(macro), AUC(macro)).
 - (Optional) Add columns “training time” and “prediction time”.
- 10. **Brief conclusion**
 - Indicate which model gives the best balance of metrics (not only Accuracy, but also F1(macro)/AUC).
 - Explain possible reasons (for example, KNN is sensitive to scaling, a tree may overfit without depth restriction, LR is a linear decision boundary).

What to submit:

- Notebook/script with the full cycle (data → training → metrics → plots).
- Plots: confusion matrix, ROC curves (OvR) and axis labels/legend.
- Summary metrics table + short textual conclusion (5–8 sentences).

Task 2. Feature Engineering and Dimensionality Reduction 25 БАЛЛОВ

Objective: Study which features most affect classification quality in a Decision Tree model, and analyze how changing hyperparameters affects the accuracy of KNN and Decision Tree models.

Steps for completion:

1. Data loading and preparation

1. Load the Iris dataset from the `sklearn.datasets` library.
2. Split the data into training and test sets (e.g., 80% / 20%) while maintaining class proportions (stratification).
3. If necessary, perform feature scaling (especially important for KNN so that all features have equal contribution).

2. Analysis of feature importance

1. Train a Decision Tree Classifier on the training data.
2. Extract feature importance values that show which characteristics most influence the model's decision.
3. Visualize feature importance as a horizontal bar chart.
4. Analyze the results: which features have the greatest influence and why they are more important than others.

3. Confusion Matrix

1. Use the best model (for example, the one that showed the highest accuracy in Task 1).
2. Plot the confusion matrix to see which classes the model identifies correctly and which it confuses.
3. Display the matrix as a heatmap with class labels on the axes.
4. Analyze:
 - which types of errors occur more often;
 - which classes the model distinguishes best;
 - whether there are cross-errors between similar classes (e.g., versicolor and virginica).

4. Study of hyperparameter influence

a) For KNN

1. Conduct a series of experiments changing the number of neighbors k (e.g., from 1 to 20).
2. For each k , compute Accuracy on the test set.
3. Plot Accuracy vs k .
4. Draw a conclusion:
 - at small k the model may overfit;
 - at too large k sensitivity is lost and quality decreases;
 - determine the optimal k range where accuracy is most stable.

b) For Decision Tree

5. Conduct experiments changing tree depth `max_depth` (e.g., from 1 to 10).
6. For each depth, measure accuracy on the training and test sets.
7. Plot two curves on one axis: train Accuracy and test Accuracy.
8. Analyze:
 - at small depth the tree underfits;
 - at too large depth the tree memorizes training data (overfits);
 - find the optimal depth with the best balance.

5. Interpretation and conclusions

1. Compare results across all visualizations.
2. Describe which features turned out to be the most significant and why.
3. Indicate which hyperparameter values gave the best result.
4. Make an overall conclusion:
 - which models are more robust to parameter changes;

- which features and settings give the best classification quality.

Expected results:

- Feature importance bar chart.
- Confusion Matrix heatmap.
- Accuracy vs number of neighbors (KNN) plot.
- Accuracy vs tree depth (Decision Tree) plot.

Task 3. Implement a Simple Perceptron (PyTorch or TensorFlow) 25 баллов

Objective: implement a basic neural network.

Task:

1. Build a simple neural network with one hidden layer (`nn.Linear` or `Dense`).
2. Use different activation functions: ReLU, Sigmoid, Tanh.
3. Compare them by accuracy on the same dataset (e.g., Iris or Breast Cancer).
4. Plot Accuracy as a function of activation choice.

Steps for completion

1. **Data loading and preparation**
 - Choose a dataset (Iris / Breast Cancer).
 - Split into train/valid/test (60/20/20).
 - Fit `StandardScaler` on train, apply to valid and test.
 - Save dataset sizes and class balance.
2. **Experiment configuration**
 - Fix seeds.
 - Define a single hyperparameter config (lr, batch size, epochs, hidden_dim, weight_decay).
 - Unified weight initialization/logic for saving best weights.
3. **Model implementation (one hidden layer)**
 - **PyTorch:** `nn.Linear(input_dim, hidden_dim) → activation → nn.Linear(hidden_dim, out_dim)`.
 - **TensorFlow:** `Dense(hidden_dim, activation=None) → manual activation → Dense(out_dim)` (without activation if you use `from_logits`).
 - Expose the activation as a parameter ("relu" | "sigmoid" | "tanh") so the loop over activations doesn't duplicate code.
4. **Training (unified loop for each activation)**
 - Forward pass → loss → backprop → optimizer step.
 - Collect histories `train_loss`, `valid_loss`, `train_acc`, `valid_acc` by epochs.
 - Early stopping by valid loss; save and restore best weights.
5. **Evaluation**
 - For each activation:

- Load best weights by valid.
 - Compute test accuracy, precision, recall, F1, plot confusion matrix.
 - Save the result in a summary table.
6. **Visualization**
- Learning curves: loss and accuracy vs epochs for each activation (two figures or two axes — two separate figures allowed).
 - Bar chart: test accuracy for ReLU, Sigmoid, Tanh (mandatory by the task).
 - (Optional) t-SNE/UMAP on hidden-layer embeddings (to analyze separability) — bonus.
7. **Fair comparison**
- Emphasize that all settings except the activation function are identical.
 - If you change anything else — justify and document it.
8. **Report (short, 1–2 pages)**
- Table with test metrics: Accuracy, Precision, Recall, F1 for each activation.
 - Graphs: learning curves + bar chart of accuracy by activations.
 - Brief analysis:
 - Which activation converged faster (by loss/acc curves)?
 - Which gave the best test result?
 - Observations on stability (if multiple runs were done).
 - Hypotheses: why so (gradients, saturation, piecewise linearity, etc.)?
 - Conclusions and recommendations (which activation to choose for similar tabular tasks and why).

What to submit (artifacts)

1. Code (.ipynb):
 - Clean end-to-end run (data → training → evaluation → plots).
 - Parameters and seeds placed in a config/top of the notebook.
2. Plots:
 - Learning curves (loss/acc) for each activation.
 - Bar chart of test accuracy by activations.
3. Metrics table on test (csv/markdown).

Task 4. MLP on MNIST or Fashion-MNIST 25 баллов

Objective: Train a multilayer perceptron (MLP) on images of handwritten digits or clothing items and study the effect of training parameters — number of epochs and batch size — on model quality.

Task:

1. Load the MNIST or Fashion-MNIST dataset
from `torchvision.datasets` / `tf.keras.datasets`.
2. Build an MLP with 2–3 hidden layers.
3. Vary training parameters:
 - Number of epochs: 5, 10, 20
 - Batch size: 32, 64, 128
4. Evaluate Accuracy and Loss for each parameter combination.

5. Visualize:
 - Examples of correct and incorrect predictions (3–5 images each).
6. Draw a conclusion on how epochs and batch size affect quality.

Steps for completion

1. **Data loading and preparation**
 1. Choose the dataset — MNIST (digits 0–9) or Fashion-MNIST (clothing categories).
 2. Split the data into train and test (usually 60,000 / 10,000 examples).
 3. Normalize pixel values (convert to the range 0–1 or $-1 \dots 1$).
 4. If necessary, create a validation part (e.g., 10% of train).
2. **Building the MLP architecture**
 1. Define the input size = number of pixels in an image ($28 \times 28 = 784$).
 2. Create a network with 2–3 hidden layers (e.g., $256 \rightarrow 128 \rightarrow 64$ neurons).
 3. Use ReLU as the activation function, and Softmax at the output for classification into 10 classes.
 4. Define the loss function (e.g., CrossEntropy Loss) and optimizer (Adam or SGD).
3. **Experiment preparation**
 1. Fix all hyperparameters except the two you will vary:
 - number of epochs (5, 10, 20);
 - batch size (32, 64, 128).
 2. Create a table of all combinations ($3 \times 3 = 9$ experiments).
 3. For each combination, perform training and evaluation on validation or test data.
4. **Model training**
 1. For each parameter combination, run MLP training.
 2. After each epoch, save Accuracy and Loss on train and validation/test.
 3. Track training dynamics (improvement or memory overflow, overfitting).
 4. At the end of each run, record the best Accuracy and Loss values.
5. **Results analysis**
 1. Build a table with results: for each pair (epochs, batch_size) — values of Accuracy and Loss.
 2. Determine under which parameters the best balance between quality and training time is achieved.
 3. Note whether there is overfitting with long training or small batches.
6. **Prediction visualization**
 1. Select several examples from the test set (e.g., 5 correct and 5 incorrect predictions).
 2. Display these images with labels: “True label” and “Predicted label”.
 3. Analyze which classes the model recognizes accurately and where it makes mistakes (for example, confuses boots and sandals).
7. **Interpretation and conclusions**
 1. Compare results for different epochs and batch sizes.
 2. Describe how increasing epochs affects Loss and Accuracy (improvement, saturation, overfitting).
 3. Explain why a larger batch size can speed up training but worsen generalization ability.

4. Make an overall conclusion about the most effective settings for the MLP in this task.

Expected results

- Table of Accuracy and Loss for all parameter combinations.
- Training dynamics plots (Accuracy/Loss by epochs).
- Visualization of correct and incorrect examples.